

programming. In imperative programming you give the computer a series of instructions which it then performs in an order.

You might wonder, what is the alternative to telling the computer what to do? How will it do anything if you don't give it a series of steps? One alternative is something called declarative programming. In declarative programming, you don't tell the computer what to do, but what it is that you want from the computer. It then figures out how to accomplish that.

A good example of this is HTML. In HTML you do not tell a computer how to draw a title or a paragraph, you merely tell the computer what you want, and it handles the rest. If this seems a little weak to you, SQL is another example. SQL code does not focus on how to get data out of a database, instead it just describes the data you want, and the computer figures out how to fetch that data. Again, if you aren't familiar with it, you will have to look it up to understand better.

Constraint programming then is a form of declarative programming in which you simply give the computer a set of variables, provide a set of constraints that apply to those variables and from that it derives a solution.

Now this might sound absolutely magical. No algorithms, no messing with functions and classes and all that. Well, you're right that it's magical, but it's not as simple as you might think. In constraint programming, the most difficult thing is simply trying to convert a problem into a set of variables and constraints. Often you need to think quite deeply to derive constraints that might not appear at first glance.

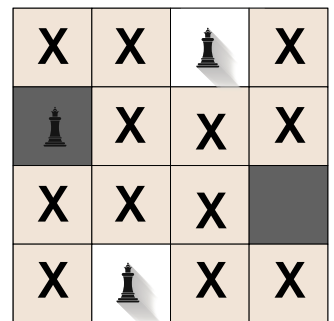
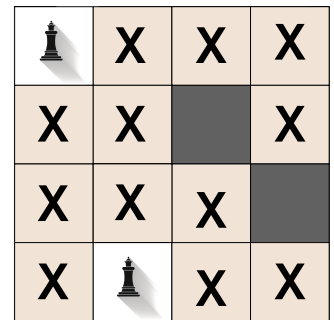
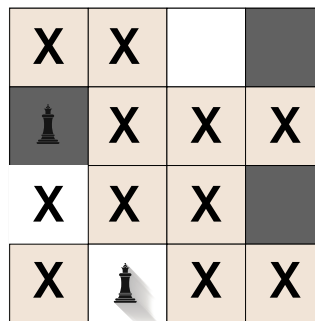
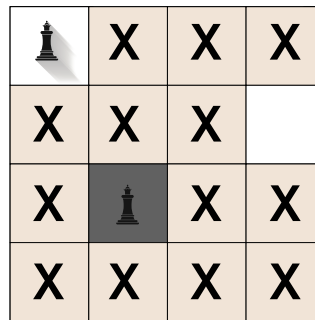
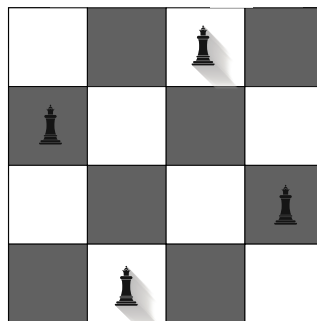
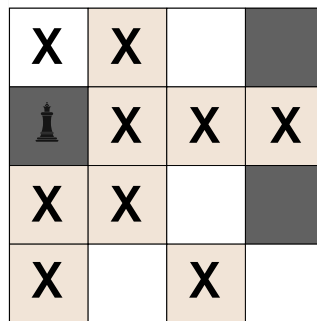
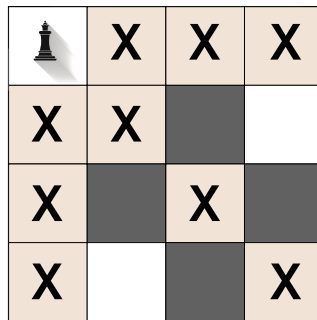
Also, constraint programming does not make a computer magically understand your natural language. It has its own syntax and conventions that differ between different programming languages and constraint programming libraries. You need to work within the constraints – see what we did there – of constraint programming systems.

It will all become clearer when we look at a few popular examples of problems that can be easily solved with constraint programming.

Example problems:

We've already mentioned Sudoku above as a good example of constraint programming, now let's look at another – decidedly academic problem called “SEND + MORE = MONEY”.

Here is how this puzzle goes. The alphabets in “SEND”, “MORE”, and “MONEY” are representations of numerical digits. So if “S” is 8, “E” is 4, “N” is 1 and “D” is 7, then “SEND” stands for the number



The n-Queens problem described in the “Solving a Constraint Program by Hand” section, is a lot tougher than it looks when you get to a larger number of queens (with a much larger grid). If you simply start placing queens randomly, you will find more often than not that you need to backtrack and try again.

The 4 queens problem illustrated on the left here shows the n-Queens problem in action with a simple version of the problem that can be solved by hand.

8417. What you need to do, is to find what each letter stands for, such that the equation “SEND + MORE = MONEY” holds true. Obviously, here a letter cannot stand for a number like 11, since it represents a digit. Another constraint is that, all alphabets stand for different digits.

If you were to look at an imperative solution for this, honestly saying, it wouldn't be hard. There are only eight distinct letters, and 10 possible values, which leads to just a few million combinations. A modern computer can chew through that pretty quickly. However such a solution would not be applicable to the next problem we discuss, and you would need to develop a solution from scratch.

Now let's see a constraint programming solution develop in this problem. First of all, we have eight variables (S, E, N, D, M, O, R, Y). The first constraint is that each letter stands for a number between 0 and 9; this is often called the domain of the variable. The second constraint is that each letter is different; there is usually a very easy way to declare this in a constraint programming environment. The final constraint is that equation declared above.

Is this all though? Another constraint that might not be apparent is that “S” and “M” can't be 0, because valid numbers can't start with a zero. Dig a bit deeper and immediately you will see that “M” is the carry from the sum of “S”, and “M”. The largest value a carry can have in this case is 1. So the maximum value of “M” is 1, which means that the value of “M” is 1 since it can't be 0. This also eliminates 1 as a solution to any other digit.

Let's look at what a solution to this problem would look like in a constraint programming language. The language we are using here is called MiniZinc, and it is in fact one of the sample codes that